

Senior Java + AWS + DevOps + Architecture

Interview Handbook — based on the supplied Job Description

How to use this handbook

For senior interviews, answer with: Problem → Design/Decision → Implementation → Failure handling → Monitoring → Result. Use real project examples wherever possible. Do not claim hands-on experience with a tool unless you can explain what you actually did with it.

Section	Focus
1. Java 11+ — Core & Coding	7 interview questions
2. Spring Boot & Hibernate	5 interview questions
3. Microservices & Architecture	5 interview questions
4. AWS	4 interview questions
5. Kubernetes & Docker	3 interview questions
6. Jenkins, CI/CD & Terraform	3 interview questions
7. Observability: ELK, InfluxDB, Grafana, Sensu	2 interview questions
8. Oracle/PostgreSQL	2 interview questions
9. Testing — Cypress/Jest	2 interview questions
10. DevOps & Production Troubleshooting	2 interview questions
11. Stakeholder Management & Product Development	3 interview questions
12. Senior Scenario / Architecture Round	2 interview questions

1. Java 11+ — Core & Coding

7 questions

Q1. What are the major Java 11+ features you would discuss in an interview?

Solution

Java 11 is an LTS release. Important interview points include the standard HTTP Client, local-variable syntax in lambda parameters, String utility methods such as `isBlank/lines/repeat/strip`, `Files.readString/writeString`, and removal of older Java EE/CORBA modules. In a senior interview, connect language features to maintainability, performance, and production use rather than listing features only.

Likely follow-up

Which Java version is your current production version and why?

Interview takeaway

Explain features you have actually used.

Q2. How does HashMap work internally?

Solution

HashMap stores key-value pairs in buckets. The hash of the key determines a bucket index. Collisions are handled through a linked structure and, in modern Java, heavily-collided buckets can be treeified. Correct `equals/hashCode` implementation is essential. Average lookup is close to $O(1)$, while pathological collisions can degrade performance.

Code example

```
Map<String, Integer> counts = new HashMap<>();
counts.put("JAVA", 1);
counts.put("SPRING", 2);
int value = counts.getOrDefault("JAVA", 0);
```

Likely follow-up

What happens if `equals()` is overridden but `hashCode()` is not?

Interview takeaway

Equal objects must produce the same hash code.

Q3. How would you make an immutable class?

Solution

Make the class `final`, fields `private` and `final`, initialize them through the constructor, provide no setters, and make defensive copies of mutable fields. Immutability makes objects easier to reason about and safer to share across threads.

Code example

```
public final class Employee {
    private final String id;
    private final List<String> skills;

    public Employee(String id, List<String> skills) {
        this.id = id;
        this.skills = List.copyOf(skills);
    }

    public String getId() { return id; }
    public List<String> getSkills() { return skills; }
```

```
}
```

Likely follow-up

Why is immutability useful in concurrent applications?

Interview takeaway

Immutable state reduces synchronization needs and accidental shared-state changes.

Q4. Stream: remove duplicate employees by ID.

Solution

Use a map keyed by the unique identifier when uniqueness is the real business rule. If the requirement is simply object equality, `distinct()` can be used, but it depends on correctly implemented `equals/hashCode`.

Code example

```
List<Employee> unique = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Function.identity(),
        (first, second) -> first
    ))
    .values()
    .stream()
    .toList();
```

Likely follow-up

How would you preserve the first occurrence and ordering?

Interview takeaway

Use `LinkedHashMap` as the collector map supplier.

Q5. CompletableFuture vs Future?

Solution

`Future` provides a handle for asynchronous computation but commonly forces blocking with `get()`. `CompletableFuture` supports composition, callbacks, combination of asynchronous operations, and explicit exception handling. In service orchestration, `CompletableFuture` is useful when independent remote calls can run concurrently.

Code example

```
CompletableFuture<String> user =
    CompletableFuture.supplyAsync(() -> getUser());

CompletableFuture<String> orders =
    CompletableFuture.supplyAsync(() -> getOrders());

CompletableFuture<String> result =
    user.thenCombine(orders, (u, o) -> u + " | " + o);
```

Likely follow-up

What happens when one future fails?

Interview takeaway

Handle it explicitly with `exceptionally`, `handle`, or `whenComplete` depending on the requirement.

Q6. Explain `ExecutorService` and thread-pool sizing.

Solution

ExecutorService separates task submission from thread management. CPU-bound workloads generally need a pool around the number of available processors, while I/O-bound workloads may use more threads because threads spend time waiting. The exact size must be validated with load testing and production metrics.

Code example

```
ExecutorService pool = Executors.newFixedThreadPool(10);
try {
    Future<String> f = pool.submit(() -> callRemoteService());
    System.out.println(f.get());
} finally {
    pool.shutdown();
}
```

Likely follow-up

Why should you avoid creating an unlimited number of threads?

Interview takeaway

Excess threads cause context switching, memory pressure, and can exhaust downstream resources.

Q7. What is the difference between synchronized, Lock, and ConcurrentHashMap?

Solution

synchronized is simple intrinsic locking. Lock provides more control such as timed acquisition and interruptible locking. ConcurrentHashMap provides concurrent map operations without locking the entire map for ordinary access. Choose the simplest construct that satisfies correctness and contention requirements.

Likely follow-up

Can ConcurrentHashMap contain null keys?

Interview takeaway

No; null is deliberately disallowed to avoid ambiguity in concurrent lookups.

2. Spring Boot & Hibernate

5 questions

Q8. Explain Spring Boot auto-configuration.

Solution

Spring Boot examines the classpath, configuration properties, and existing beans and conditionally creates useful infrastructure. @SpringBootApplication combines component scanning, configuration, and auto-configuration. This reduces boilerplate while still allowing explicit overrides.

Code example

```
@SpringBootApplication
public class OrderApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
```

Likely follow-up

How do you override an auto-configured bean?

Interview takeaway

Define your own bean or use the appropriate configuration property/conditional mechanism.

Q9. How do you design global exception handling in Spring Boot?

Solution

Use @RestControllerAdvice with targeted @ExceptionHandler methods. Return a consistent error contract containing timestamp, status, code, message, and correlation/request ID. Avoid exposing stack traces or internal implementation details to clients.

Code example

```
@RestControllerAdvice
class GlobalHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<ApiError> handle(ResourceNotFoundException ex) {
        return ResponseEntity.status(404)
            .body(new ApiError("RESOURCE_NOT_FOUND", ex.getMessage()));
    }
}
```

Likely follow-up

How do you distinguish validation errors from business errors?

Interview takeaway

Use separate exception types and stable API error codes.

Q10. Explain @Transactional and common pitfalls.

Solution

Transactional defines a unit of work with commit or rollback semantics. Common pitfalls include self-invocation, catching exceptions without rethrowing when rollback is expected, long-running transactions, and accessing lazy data outside the persistence context. Transaction boundaries should usually align with a business operation.

Code example

```
@Transactional
public void transfer(long from, long to, BigDecimal amount) {
    debit(from, amount);
    credit(to, amount);
}
```

Likely follow-up

When would you use readOnly=true?

Interview takeaway

For transaction semantics/optimization on read operations where supported; it does not magically make every database read faster.

Q11. What is the Hibernate N+1 problem?

Solution

N+1 occurs when one query loads parent records and then one additional query is executed for each parent to load related data. Fix it with appropriate fetch joins, entity graphs, batch fetching, or carefully designed projections. Avoid globally switching everything to EAGER.

Code example

```
@Query("""
    select o from Order o
    join fetch o.items
    where o.id = :id
""")
Optional<Order> findOrderWithItems(Long id);
```

Likely follow-up

How would you prove that N+1 exists?

Interview takeaway

Enable SQL logging/metrics and inspect query count and database traces.

Q12. Optimistic vs pessimistic locking?

Solution

Optimistic locking assumes collisions are uncommon and uses a version field. Pessimistic locking acquires database locks and is useful when contention and correctness require serialized access. Optimistic locking usually scales better but requires conflict handling.

Code example

```
@Entity
class Account {
    @Id
    private Long id;

    @Version
    private Long version;
}
```

Likely follow-up

What should the API do after an optimistic-lock conflict?

Interview takeaway

Return a controlled conflict response or retry when the operation is safely retryable.

3. Microservices & Architecture

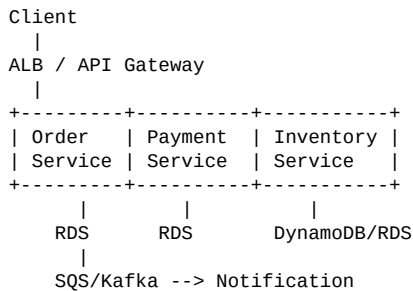
5 questions

Q13. How would you design a highly available order-management system?

Solution

Use an API gateway/load balancer in front of independently deployable services. Keep services stateless where possible, use PostgreSQL/RDS for transactional data, asynchronous messaging for decoupled workflows, and Redis or another cache where justified. Add timeouts, retries with backoff, circuit breakers, idempotency, centralized logging, metrics, tracing, and automated deployment.

Code example



Likely follow-up

How would you handle payment success but order update failure?

Interview takeaway

Use an event/outbox or Saga-style workflow with idempotent consumers and compensating actions.

Q14. What is the Saga pattern?

Solution

Saga manages a distributed business transaction as a sequence of local transactions. If a later step fails, compensating actions undo the business effect of earlier steps. It can be choreography-based through events or orchestration-based through a coordinator.

Likely follow-up

When would you prefer orchestration?

Interview takeaway

When workflow visibility and centralized control are more important than fully decentralized event choreography.

Q15. Explain circuit breaker, timeout, and retry.

Solution

Timeout prevents a caller from waiting indefinitely. Retry handles transient failures but must use bounded attempts and backoff. Circuit breaker stops repeatedly calling an unhealthy dependency and allows recovery probes later. Combining these without limits can amplify outages, so policies must be designed together.

Code example

```
// Conceptual policy
```

```
timeout = 2s
retry = 2 attempts
backoff = exponential + jitter
circuitBreaker = open after threshold failures
```

Likely follow-up

What is retry storm?

Interview takeaway

Many callers retry simultaneously and overload an already failing dependency.

Q16. What is idempotency and how would you implement it?

Solution

An operation is idempotent when repeating the same request produces the same business result. For payment/order APIs, accept an Idempotency-Key, persist its result with the request identity, and return the stored result for duplicate requests. This is critical when clients retry after timeouts.

Code example

```
POST /payments
Idempotency-Key: 9f2a-1234

// Server:
// 1. Check key
// 2. If processed, return stored result
// 3. Otherwise process transaction
// 4. Store key + result atomically
```

Likely follow-up

What database constraint helps?

Interview takeaway

A unique constraint on the idempotency key prevents duplicate processing races.

Q17. Explain CQRS and when it helps.

Solution

CQRS separates command/write models from query/read models. It can help when read and write workloads have very different scaling or data-shaping requirements. It adds complexity, synchronization concerns, and often eventual consistency, so it should be introduced for a concrete requirement rather than by default.

Likely follow-up

Is CQRS required for every microservice?

Interview takeaway

No. A conventional service/repository model is often simpler.

4. AWS

4 questions

Q18. SQS vs SNS?

Solution

SQS is a durable queue used to buffer work and decouple producers from consumers. SNS is a pub/sub notification service that can fan out messages to multiple subscribers. A common architecture is SNS topic feeding multiple SQS queues so each consumer gets independent buffering and retry behavior.

Likely follow-up

Why put SQS behind SNS?

Interview takeaway

To provide fan-out while preserving independent consumer processing and retry.

Q19. RDS vs DynamoDB?

Solution

RDS is a managed relational database suited to SQL, joins, transactions, and relational data models. DynamoDB is a managed NoSQL key-value/document database designed for predictable low-latency access at scale when the access patterns are known. Selection should follow data relationships, consistency needs, access patterns, and operational requirements.

Likely follow-up

Can DynamoDB perform joins like PostgreSQL?

Interview takeaway

No; data modeling generally denormalizes around access patterns.

Q20. How would you secure a Spring Boot application on AWS?

Solution

Use IAM roles rather than hard-coded credentials, private subnets for internal resources, security groups with least privilege, TLS, Secrets Manager/Parameter Store for secrets, encryption at rest, controlled IAM policies, audit logs, and application-level authorization such as OAuth2/JWT where applicable.

Likely follow-up

Where should database passwords be stored?

Interview takeaway

Use a managed secrets solution and inject them securely at runtime.

Q21. How would you deploy a Spring Boot application to AWS Kubernetes?

Solution

Build and test the application in Jenkins, create a versioned Docker image, push it to a registry such as ECR, deploy to EKS using Kubernetes manifests/Helm, expose it through an ingress/load balancer, configure secrets/config maps, readiness/liveness probes, autoscaling, and monitoring. Terraform can provision the surrounding infrastructure.

Code example

```
Jenkins
-> Maven test
-> Docker build
-> ECR push
-> kubectl/Helm
-> EKS Deployment
-> Service/Ingress
-> Health + Metrics
```

Likely follow-up

How do you rollback?

Interview takeaway

Use versioned images and Kubernetes rollout history/Helm release rollback with health verification.

5. Kubernetes & Docker

3 questions

Q22. Pod is in CrashLoopBackOff. How do you troubleshoot?

Solution

Start with `kubectl get pods`, then describe the pod and inspect current and previous logs. Check environment variables, ConfigMaps, Secrets, probes, resource limits, application startup failures, database/network connectivity, and exit codes. If memory is involved, inspect OOMKilled and container limits.

Code example

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
```

Likely follow-up

What does OOMKilled indicate?

Interview takeaway

The container exceeded its memory limit or was terminated due to memory pressure.

Q23. Readiness vs liveness probe?

Solution

Readiness determines whether the pod should receive traffic. Liveness determines whether the container is considered unhealthy and should be restarted. A slow-starting application should not be killed by an overly aggressive liveness probe.

Code example

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
```

Likely follow-up

What is a startup probe?

Interview takeaway

It gives slow-starting applications time to initialize before liveness checking becomes active.

Q24. Explain Docker image vs container.

Solution

An image is an immutable packaged template containing application code and runtime dependencies. A container is a running instance of that image with an isolated process environment. Tag images with immutable version identifiers to support traceability and rollback.

Code example

```
FROM eclipse-temurin:17-jre
COPY target/app.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Likely follow-up

Why should containers avoid storing important state locally?

Interview takeaway

Containers are ephemeral; persistent state belongs in managed storage.

6. Jenkins, CI/CD & Terraform

3 questions

Q25. Design a Jenkins CI/CD pipeline for Java microservices.

Solution

The pipeline should checkout source, compile, run unit tests, run static/security checks, package the artifact, build and scan a Docker image, push it to a registry, deploy to a non-production environment, run smoke/integration tests, require an approval where appropriate, deploy to production, and verify health. Every deployment should be traceable to a Git commit/image version.

Code example

```
pipeline {
  stages {
    stage('Build') { steps { sh './mvnw clean verify' } }
    stage('Docker') { steps { sh 'docker build -t app:$BUILD_NUMBER .' } }
    stage('Push') { steps { sh 'docker push registry/app:$BUILD_NUMBER' } }
    stage('Deploy') { steps { sh 'helm upgrade --install app chart/' } }
  }
}
```

Likely follow-up

How do you prevent bad deployments?

Interview takeaway

Automated tests, quality gates, security scans, health checks, approvals for sensitive environments, and rollback.

Q26. What is Infrastructure as Code?

Solution

IaC means defining infrastructure in version-controlled, repeatable configuration rather than manually creating resources. Terraform is declarative: you describe desired state and Terraform plans the changes. This improves repeatability, reviewability, and environment consistency.

Code example

```
terraform init
terraform plan
terraform apply

# Typical structure:
main.tf
variables.tf
outputs.tf
modules/
```

Likely follow-up

What is Terraform state?

Interview takeaway

State maps declared resources to real infrastructure so Terraform can calculate changes. Protect it and use remote state/locking for teams.

Q27. How would you implement one-click deployment?

Solution

A Jenkins pipeline can accept environment, application version, and deployment parameters. It validates inputs, retrieves the immutable artifact, applies infrastructure changes when required, deploys the selected version, runs health/smoke tests, and reports the result. The process should be auditable and repeatable.

Likely follow-up

What makes one-click deployment safe?

Interview takeaway

Immutable artifacts, automated validation, approvals where needed, health checks, and reliable rollback.

7. Observability: ELK, InfluxDB, Grafana, Sensu

2 questions

Q28. Explain logs vs metrics and how ELK/Grafana fit.

Solution

Logs provide detailed event context; ELK commonly centralizes and searches logs, with Kibana used for visualization/search. Metrics are numeric time-series measurements such as latency, CPU, error rate, and throughput; InfluxDB can store time-series data and Grafana can visualize dashboards and alerts. Monitoring/alerting tools such as Sensu can evaluate health signals and trigger actions.

Likely follow-up

What are the four golden signals?

Interview takeaway

Latency, traffic, errors, and saturation.

Q29. Production API latency increased from 200 ms to 5 seconds. How do you investigate?

Solution

First determine scope and start time using dashboards. Compare latency, traffic, error rate, CPU, memory, GC, thread pools, connection pools, database latency, and downstream calls. Correlate logs with request IDs and inspect slow queries. Reproduce safely, identify the bottleneck, fix it, load-test it, deploy gradually, and monitor.

Likely follow-up

What if CPU and memory look normal?

Interview takeaway

Investigate DB latency, thread blocking, connection pool exhaustion, network latency, and slow downstream dependencies.

8. Oracle/PostgreSQL

2 questions

Q30. How do you optimize a slow SQL query?

Solution

Check the actual execution plan, row estimates, join strategy, index usage, filtering, sorting, and returned row count. Ensure predicates are index-friendly, avoid unnecessary columns, validate statistics, and compare performance before and after. Never add indexes blindly because writes and storage also have costs.

Code example

```
EXPLAIN ANALYZE
SELECT customer_id, COUNT(*)
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY customer_id;
```

Likely follow-up

What is a composite index?

Interview takeaway

An index over multiple columns; column order matters for the access patterns.

Q31. How do you handle database connection-pool exhaustion?

Solution

Check pool metrics, active/idle connections, wait time, transaction duration, slow queries, leaked connections, and database max connections. Fix the root cause before simply increasing pool size. Keep transactions short and ensure resources are released. Align pool sizes with database capacity and total application replicas.

Likely follow-up

Why can increasing the pool make things worse?

Interview takeaway

Too many concurrent connections can overwhelm the database and increase contention.

9. Testing — Cypress/Jest

2 questions

Q32. How would you test an Angular UI with Cypress?

Solution

Use Cypress for end-to-end user journeys: login, navigation, form validation, API-driven flows, and critical business scenarios. Stub unstable external APIs where appropriate, use deterministic test data, and keep selectors stable using dedicated data attributes.

Code example

```
describe('Order flow', () => {
  it('creates an order', () => {
    cy.visit('/orders');
    cy.get('[data-testid="create-order"]').click();
    cy.get('[data-testid="product"]').select('P100');
    cy.get('[data-testid="submit"]').click();
    cy.contains('Order created').should('be.visible');
  });
});
```

Likely follow-up

What should not be covered only by Cypress?

Interview takeaway

Core business logic should also have fast unit/component tests.

Q33. Jest vs Cypress?

Solution

Jest is commonly used for fast unit/component-level testing and mocking. Cypress is focused on browser-based end-to-end and integration-style flows. They complement each other rather than replace one another.

Likely follow-up

How do you reduce flaky tests?

Interview takeaway

Stable selectors, controlled data, explicit waits based on application state, isolated environments, and deterministic dependencies.

10. DevOps & Production Troubleshooting

2 questions

Q34. A Jenkins deployment is green but the application is failing. What do you do?

Solution

Separate pipeline success from application health. Check deployment version, Kubernetes rollout status, pod logs, events, probes, configuration, secrets, service endpoints, ingress, and downstream dependencies. Verify the deployed artifact matches the intended Git commit. If impact is high, rollback to the last known-good immutable version while investigating.

Likely follow-up

What should Jenkins do after deployment?

Interview takeaway

Run automated health/smoke tests and fail the deployment if the application does not become healthy.

Q35. How would you investigate a memory leak in a Java application?

Solution

Establish whether heap usage grows continuously after GC. Inspect JVM metrics and GC logs, take heap dumps at safe points, analyze retained objects and dominator trees, and correlate growth with traffic/deployment changes. Common causes include unbounded caches, static collections, listeners, thread-local misuse, and retained request data. Fix, load-test, and monitor after deployment.

Likely follow-up

How do you distinguish a leak from normal heap growth?

Interview takeaway

Compare post-GC baseline over time; a rising post-GC baseline is a strong signal.

11. Stakeholder Management & Product Development

3 questions

Q36. How do you handle conflicting stakeholder requirements?

Solution

Clarify the business objective, identify constraints, document options and trade-offs, quantify impact where possible, and facilitate a decision with the appropriate product/business owner. Once decided, record acceptance criteria and communicate scope changes to engineering and QA.

Likely follow-up

What if the stakeholder asks for an urgent production change?

Interview takeaway

Assess risk, follow change controls, define rollback, test the smallest safe change, and communicate impact.

Q37. Tell me about your product development experience.

Solution

Use a concrete product example and describe the full lifecycle: requirement discovery, architecture, backlog refinement, development, code review, testing, CI/CD, production deployment, monitoring, incident handling, and iterative improvements. Emphasize measurable technical or business outcomes without exaggerating ownership.

Likely follow-up

How did you decide which technical debt to address?

Interview takeaway

Balance risk, customer impact, delivery commitments, operational cost, and long-term maintainability.

Q38. How do you conduct an effective code review?

Solution

Review correctness first, then security, error handling, performance, API compatibility, maintainability, tests, observability, and operational impact. Ask questions rather than making personal judgments. Keep standards consistent and automate checks such as formatting, static analysis, and security scanning.

Likely follow-up

What should never be left only to manual review?

Interview takeaway

Basic formatting, many static rules, dependency/security checks, and repeatable tests.

12. Senior Scenario / Architecture Round

2 questions

Q39. Design a one-click, highly available deployment platform for Spring Boot services on AWS.

Solution

A practical design uses Git as the source of truth, Jenkins for orchestration, Maven/Gradle for build, unit/integration tests and quality/security gates, Docker for immutable images, ECR for storage, Terraform for AWS infrastructure, and EKS for deployment. Use Kubernetes Deployments with readiness/liveness probes, HPA, rolling releases, and ingress. Store secrets in AWS Secrets Manager/Parameter Store. Send application logs to ELK and metrics to InfluxDB/Grafana; use monitoring/alerting to detect failures. Add smoke tests after deployment and automated rollback on failed health checks.

Code example

```
Git
|
Jenkins
|-- Build/Test/Scan
|
Docker -> ECR
|
Terraform -> AWS infrastructure
|
EKS
|-- Deployment -> Pods
|-- Service -> Ingress/ALB
|
+--> Logs -> ELK/Kibana
+--> Metrics -> InfluxDB/Grafana
+--> Alerts -> Monitoring/Sensu
```

Likely follow-up

What would you document in a C4 architecture diagram?

Interview takeaway

Context, containers, components, and—where useful—code-level relationships; keep each level focused on a clear audience.

Q40. How would you answer 'Why should we hire you for this role?'

Solution

Connect your experience to the role's technical needs: strong Java/Spring development, production issue resolution, full-stack understanding, database experience, CI/CD exposure, architecture thinking, and ability to work with stakeholders. Give two or three concrete examples instead of making broad claims. Be accurate about the depth of AWS/Kubernetes/Terraform experience and explain where you have hands-on versus conceptual knowledge.

Likely follow-up

What is your strongest technical area?

Interview takeaway

Choose the area where you can demonstrate the deepest real-world examples and follow-up detail.

Final 24-Hour Revision Checklist

- ☐ Java: HashMap, equals/hashCode, streams, concurrency, CompletableFuture, JVM/GC.
- ☐ Spring Boot: REST, validation, exception handling, transactions, security, JPA/Hibernate.
- ☐ Architecture: microservices, Saga, idempotency, resilience, caching, HLD/LLD, C4.
- ☐ AWS: IAM, VPC, S3, SQS, SNS, RDS, DynamoDB, EKS, CloudWatch, Secrets.
- ☐ Kubernetes: Pod, Deployment, Service, Ingress, ConfigMap, Secret, probes, HPA, rollback.
- ☐ CI/CD: Git → Jenkins → test/scan → Docker → registry → Terraform → Kubernetes → smoke test.
- ☐ Terraform: state, modules, variables, plan/apply, remote state and locking.
- ☐ Observability: logs vs metrics, ELK/Kibana, InfluxDB/Grafana, alerts and golden signals.
- ☐ Databases: execution plans, indexes, transactions, isolation, locking, connection pools.
- ☐ Testing: Jest unit/component testing and Cypress E2E.
- ☐ Behavioral: production incident, stakeholder conflict, architecture decision, code review, mentoring.
- ☐ Prepare 3 real stories: production issue fixed, architecture/design decision, successful delivery/product feature.

Interview answer formula

For scenario questions: Context → Problem → Investigation → Root cause → Options considered → Chosen solution → Implementation → Testing → Deployment → Monitoring → Result → What you would improve next time.